

MATH 231 – Linear Algebra I

Setup Guide 1 ■ Git and GitHub

Queens College, CUNY ■ Fall 2026 ■ Instructor: Kevin Bedoya

What this guide does. By the end you will have a private GitHub repository named `QC_MATH231`, a copy of it on your own computer, three folders inside it for your coursework, and the ability to send your work to GitHub with two commands. This is how you will hand in every homework, project, and extra-credit assignment this semester.

No experience assumed. If you have never opened a terminal, that is fine — every command you need is written out. Follow the steps in order. Each one ends with a **Checkpoint** telling you what you should see if it worked. If something goes wrong, §10 is a troubleshooting guide covering the problems that actually happen.

Everything here is free. You will not be asked to pay for anything or enter a credit card.

1. What git and GitHub are

Two different things with confusingly similar names.

- **git** is a program that runs on *your* computer. It watches a folder and records snapshots of it over time. Each snapshot is called a **commit**, and it comes with a short message saying what changed. Nothing is ever silently overwritten — you can always look back at what a file used to be.
- **GitHub** is a website that stores copies of those folders online. It is where your work goes so that I can see it, and where my feedback comes from so that you can see it.

A rough analogy: git is like the version history in Google Docs, and GitHub is like Google Drive. The important difference is that git only records a snapshot when you *tell* it to — it does not autosave.



That picture is the whole workflow. Sections 6 and 7 walk through it.

2. Create a GitHub account

Skip this if you already have one.

1. Go to <https://github.com> and click **Sign up**.
2. Enter an email address, a password, and a username. Your username is public and hard to change later — pick something you would not mind me or a future employer seeing.
3. Verify your email address when GitHub sends the confirmation.

Checkpoint. You can sign in at <https://github.com> and see your profile page.

Watch out. Write your username and password down somewhere safe. You will need them in §5, and a forgotten password is the single most common reason a student cannot submit on time.

3. Create the private repository

A **repository** — “repo” for short — is just a folder that git tracks.

1. Sign in to GitHub. Click the + in the top-right corner, then **New repository**.

2. **Repository name:** type exactly

QC_MATH231

Note the underscore, and that there are no spaces.

3. **Description:** optional. Something like “MATH 231 coursework” is fine.

4. **Visibility:** choose **Private**. This matters — your coursework should not be public, both for your privacy and so that others cannot copy your solutions.

5. Under *Initialize this repository with*, tick **Add a README file**. This creates a starting file so the repository is not empty, which makes the next steps simpler.

6. Click **Create repository**.

Checkpoint. You are looking at a page titled your-username/QC_MATH231 with a *Private* label next to the name, and it contains one file called README.md.

4. Install git

You need the git program on your own computer. Find your operating system below.

Windows

The easiest route is the official installer.

1. Go to <https://git-scm.com/download/win> and let the download start.
2. Run the installer. **Accept every default** by clicking *Next* through all the screens — the defaults are correct and the options are not worth reading on a first install.
3. When it finishes, open the **Start** menu, type **Git Bash**, and open it. This is a terminal window; you will type your git commands here for the rest of the semester.

If you prefer, you can install from a terminal instead:

```
winget install --id Git.Git -e --source winget
```

macOS

Open the **Terminal** app (press Cmd+Space, type **Terminal**, press Enter) and run:

```
git --version
```

If git is missing, macOS will pop up a window offering to install the Command Line Developer Tools. Click **Install** and wait. If no window appears, run this instead:

```
xcode-select --install
```

Linux

Open a terminal and run the line for your distribution:

```
sudo apt update && sudo apt install git      # Ubuntu, Debian, Mint
sudo dnf install git                         # Fedora
sudo pacman -S git                          # Arch
```

Check that it worked

In your terminal (Git Bash on Windows), run:

```
git --version
```

Checkpoint. You see something like `git version 2.43.0`. The exact number does not matter.

Tell git who you are

Git stamps your name and email onto every commit. Set them once, now, or your first commit will fail with a complaint. Run both lines, substituting your own details:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Watch out. Use the *same* email address you signed up to GitHub with. Otherwise your commits will not be linked to your account.

5. Clone the repository

Cloning means downloading a copy of the GitHub repository onto your computer, already set up so git knows where it came from.

1. On your repository page on GitHub, click the green **Code** button.
2. Make sure the **HTTPS** tab is selected, then click the copy icon to copy the address. It looks like `https://github.com/your-username/QC_MATH231.git`
3. In your terminal, move to wherever you want the folder to live. For example, to put it on your Desktop:

```
cd Desktop
```

4. Clone it, pasting the address you copied:

```
git clone https://github.com/your-username/QC_MATH231.git
```

5. Move into the new folder:

```
cd QC_MATH231
```

Checkpoint. Running `ls` (or `dir` on Windows outside Git Bash) shows `README.md`. Running `git status` says “On branch main, nothing to commit, working tree clean.”

Watch out. If git asks for a **password**, do not type your GitHub password — GitHub stopped accepting those in 2021. You need a *personal access token* instead. §10.2 explains how to make one; it takes about a minute.

6. Create your folders

Inside the repository, make the three folders where your work will go. From inside `QC_MATH231`, run:

```
mkdir homework projects extra-credit
```

Now a quirk that catches everybody: **git tracks files, not folders**. An empty folder is invisible to git and will never appear on GitHub. The standard fix is to put one placeholder file in each. On Git Bash, macOS, or Linux:

```
touch homework/.gitkeep projects/.gitkeep extra-credit/.gitkeep
```

The name `.gitkeep` has no special meaning to git — it is just a conventional name for “this file exists only so the folder does”. Once you put real assignments in these folders you can leave the placeholders or delete them; it makes no difference.

Checkpoint. Running `git status` now lists `extra-credit/`, `homework/` and `projects/` under *Untracked files*.

7. Save your work and send it to GitHub

This is the loop you will repeat all semester. Four commands, in order.

7.1. See what changed — `git status`

```
git status
```

This changes nothing; it only reports. It tells you which files you have edited, created, or deleted. Run it whenever you are unsure what state you are in — it is the safest command in git.

7.2. Choose what to save — `git add`

```
git add .
```

The dot means “everything in this folder”. This does not save anything yet; it marks files as *ready* to be saved. (You can also name one file, as in `git add homework/hw1.py`, but the dot is fine for this course.)

7.3. Save the snapshot — `git commit`

```
git commit -m "Add coursework folders"
```

The `-m` flag supplies the **message**, in quotes: a short description of what you did. The message is required. Write something a human could read later — "Finish homework 1" is useful, "stuff" is not.

Checkpoint. Git prints a summary like `[main a1b2c3d] Add coursework folders` followed by a count of files changed.

7.4. Upload it — `git push`

```
git push
```

This sends your commits to GitHub. Until you push, your work exists only on your own computer — and only pushed work can be graded.

Checkpoint. Refresh your repository page on GitHub in a browser. You now see the homework, projects and extra-credit folders, and your commit message next to them.

Command	What it does
<code>git status</code>	shows what has changed; changes nothing
<code>git add .</code>	marks all changes to be saved
<code>git commit -m "message"</code>	saves a snapshot, with a description
<code>git push</code>	uploads your saved snapshots to GitHub
<code>git pull origin main</code>	downloads changes from GitHub (§9)

Those five are all you need. Git has hundreds of other commands and you can ignore every one of them for this course.

8. Add me as a collaborator

Your repository is private, so by default nobody but you can see it — me included. Adding me as a **collaborator** lets me read your submissions, grade them, and push written feedback back into your repository.

1. On your repository page, click **Settings** (in the row of tabs near the top, on the right).
2. In the left sidebar, click **Collaborators**. GitHub may ask you to confirm your password.
3. Click **Add people**.
4. Type my username exactly:

KevBedoya

5. Select me from the results and click **Add KevBedoya to this repository**.

Checkpoint. Under *Manage access* you see my username listed as *Pending invite*, which becomes *Collaborator* once I accept.

Watch out. Do this *before* the first assignment is due. If I am not a collaborator I cannot see your work at all, and a submission I cannot see is a submission that cannot be graded.

9. Getting my feedback — `git pull`

When I grade your work I will push comments and corrections directly into your repository. Those changes exist on GitHub but not yet on your computer. To bring them down, run this from inside your `QC_MATH231` folder:

```
git pull origin main
```

Read it as: *pull from the remote called **origin** (which is GitHub) the branch called **main***. Git downloads whatever is new and merges it into your local copy.

Make a habit of running `git pull origin main` **before** you start working, not after. If you and I have both changed things, pulling first keeps them separate and avoids the conflict described in §10.9.

Checkpoint. Git either reports `Already up to date.` or lists the files it just downloaded.

10. Troubleshooting

These are the problems that actually come up. Find your error message.

10.1. “git” is not recognised / command not found

```
'git' is not recognized as an internal or external command
bash: git: command not found
```

Git either is not installed, or your terminal was open before you installed it and has not noticed. **Close the terminal completely and open a new one**, then try `git --version` again. If it still fails, redo §4. On Windows, make sure you are using **Git Bash** and not Command Prompt.

10.2. It asks for a password and rejects the right one

GitHub no longer accepts your account password from the command line. You need a **personal access token**, which is a long password generated for one purpose.

1. On GitHub, click your profile picture (top right) → **Settings**.
2. Scroll to the very bottom of the left sidebar → **Developer settings**.
3. **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
4. Give it a note like `MATH 231`, set an expiry (a year is fine), and tick the **repo** checkbox.
5. Click **Generate token**, then **copy it immediately** — GitHub will never show it to you again.
6. When git asks for a password, paste the token instead.

Store the token somewhere safe, like a password manager or a note on your phone.

10.3. “Please tell me who you are”

```
*** Please tell me who you are.
```

You skipped the configuration at the end of §4. Run the two `git config --global` lines and commit again.

10.4. My folder does not appear on GitHub

It is empty, and git does not track empty folders. Put a file in it — see the `.gitkeep` step in §6 — then add, commit, and push again.

10.5. “fatal: not a git repository”

You are running a git command from the wrong place. Git commands only work inside a cloned repository. Run:

```
cd QC_MATH231
```

and try again. Use `pwd` to see which folder you are currently in.

10.6. “nothing to commit, working tree clean”

Git sees no changes. Usually this means you forgot to *save* the file in your editor, or you already committed the change. Check with `git status`.

10.7. “Updates were rejected”

```
! [rejected]  main -> main (fetch first)
```

GitHub has commits that your computer does not — typically feedback I pushed. Download them first, then push again:

```
git pull origin main
git push
```

10.8. It opened a strange full-screen text editor

You ran `git commit` without `-m`, so git opened an editor to ask for the message. If it looks like a plain screen with tildes down the left, that is Vim: press `Esc`, then type `:q!` and press Enter to escape. Then commit properly with the message on the command line:

```
git commit -m "your message here"
```

10.9. Merge conflict

If a pull reports a *conflict*, it means the same lines were changed in two places and git cannot decide which to keep. Do not guess and do not delete the folder. Take a screenshot of the message and

contact me — these are quick to fix when someone talks you through them, and this course will rarely produce one.

10.10. Still stuck

Email me at `Kevin.Bedoya32@stu-mail.qc.cuny.edu` or ask on the class Discord. Include:

- your operating system,
- the exact command you typed, and
- the full error message, copied as text or as a screenshot.

Nobody is expected to arrive knowing this. Ask early — a setup problem the night before a deadline is a setup problem solved too late.